

KN-IG: 정책 기반 파일 무결성 보호 시스템

강지운*, 김주환*, 김태우*, 나원현*, 이재흥**

KN-IG : A Policy-Based File Integrity Guard System

Kang Jiwoon*, Kim Juhwan*, Kim Taewoo*, Na Wonhyun*, Lee Jaeheung**

요 약

산업용 OT(Operational Technology) 환경에서는 파일 및 설정 정보의 무결성 훼손이 생산 중단, 설비 오동작, 안전 사고로 이어질 수 있으므로, 사후 탐지 중심의 기존 무결성 감시 체계만으로는 한계가 있다. 본 논문은 Linux 기반 OT 호스트를 위한 정책 기반 파일 무결성 보호 시스템 KN-IG를 제안한다. KN-IG는 kernel hook을 통한 차단, mTLS 기반 중앙 데이터 수집을 통합한 Agent 중심 구조를 갖는다. 또한 inode 기반 정책 식별과 lock/maintenance 이중 운용 모드를 통해 비인가 변경 차단과 유지보수, 감사, 추적을 함께 지원한다. 더불어 LKM310, LKM42, LKM415, eBPF LSM으로 이어지는 kernel modules 적용 전략을 제시하여 서로 다른 Linux kernel 세대에서도 동일한 상위 정책 의미를 유지하도록 설계하였다. 제안 방식은 OT 운영 연속성을 고려한 무결성 보호 체계로서, 기존 탐지 중심 FIM 대비 사전 차단, 행위 맥락 보존 측면의 확장 가능성을 제공한다.

Abstract

In industrial Operational Technology (OT) environments, the compromise of file and configuration integrity can lead to production stoppages, equipment malfunctions, and safety accidents; therefore, existing integrity monitoring systems focused on post-detection alone have limitations. This paper proposes KN-IG, a policy-based file integrity protection system for Linux-based OT hosts. KN-IG features an agent-centric architecture that integrates blocking via kernel hooks and central data collection based on mTLS. Furthermore, it supports unauthorized change blocking, maintenance, auditing, and tracing through inode-based policy identification and a dual lock/maintenance operation mode. In addition, it presents an adaptation strategy for kernel modules extending from LKM310, LKM42, LKM415, to eBPF LSM, ensuring that the same high-level policy semantics are maintained across different Linux kernel generations. As an integrity protection system designed for OT operational continuity, the proposed approach offers scalability in terms of proactive blocking and behavioral context preservation, compared to existing detection-centric FIMs.

Key words

Operational Technology, Linux Kernel, eBPF, Integrity Protection, OT Security

*경기대학교 AI 컴퓨터공학부 컴퓨터공학전공, SW안전보안전공, 전자공학과

**경기대학교 소프트웨어경영대학 이재흥 교수 jhlee@kyonggi.ac.kr

※ 본 연구는 과학기술정보통신부 및 정보통신기획평가원의 SW중심대학사업의 연구결과로 수행되었음(2021-0-01393)

I. 서 론

IT (Information Technology)는 주로 데이터의 생성, 저장, 처리, 전송과 같은 정보 중심 기능을 수행하는 시스템을 의미하는 반면, OT(Operation Technology)는 산업장비와 물리 센서, PLC(Programmable Logic Controller), HMI(Human-Machine Interface), RTU (Remote Terminal Unit), 산업용 PC와 같이 물리 설비를 감시하고 제어하여 실제 공정 상태를 변화시키는 시스템을 의미한다. 따라서 OT 환경의 보안 사고는 정보 훼손에 그치지 않고 생산 설비의 상태 변화, 공정 정지, 안전기능 저하로 이어질 수 있다.

보안 목표의 우선순위 역시 IT와 OT 사이에는 차이점이 존재한다. 일반적으로 IT 보안에서는 기밀성을 가장 우선시하는 반면, OT 보안에서는 공정 중단이나 비정상 제어가 곧 생산 손실과 안전 문제를 유발할 수 있으므로 가용성을 우선순위로 둔다. NIST(National Institute of Standards and Technology)는 OT 보안에서 가용성, 성능, 신뢰성, 안전 요구를 함께 충족해야 하며, 특히 OT에서는 순간적인 재부팅이나 짧은 다운타임조차 허용되기 어렵다고 설명한다[1].

종합하면, IT환경과 OT환경은 운영 목적과 시스템 구성, 요구되는 가용성 수준 등에서 본질적인 차이를 지니며, 이러한 특성은 두 영역의 네트워크를 분리하여 운영하는 주요한 이유가 된다. 일반적인 OT 환경은 외부 접근이 엄격히 통제되는 폐쇄적인 구조를 기반으로 운영됐으며, 이에 따라 보안의 중요성이 IT 환경에 비해 상대적으로 낮게 인식되어 왔다. 그러나 최근 산업 전반의 디지털 전환이 가속화되고 IT 환경과의 연계가 확대됨에 따라, OT 환경을 대상으로 하는 사이버 공격이 지속적으로 증가하고 있다. 이로 인해 외부 공격으로 인한 피해 규모 또한 점차 확대되는 추세를 보이며, OT 환경에 대한 체계적인 보안 대응의 필요성이 더 강조되고 있다.

최근 MES(Manufacturing Execution System), ERP(Enterprise Resource Planning), Historian, 원격 유지보수 시스템, 클라우드 기반 분석 환경이 현장 제어 자산과 연계되면서 IT와 OT의 융합이 빠르게

진행되고 있다. 이에 따라 리눅스 기반 HMI, RTU, 게이트웨이, 산업용 서버는 운영 효율성과 유연성을 높이고 있다. 그러나 이러한 통합 운영 환경은 기업 망에서 발생한 침해가 운영망의 설정 파일, 기동 스크립트, 제어 로직, 서비스 구성에 영향을 미칠 수 있는 공격 경로를 확대하고 있다. NIST는 산업 제어시스템이 점점 더 빈번하고 정교한 공격에 직면하고 있으며, 이러한 공격이 운영과 작업자 안전을 동시에 위협할 수 있음을 지적한다.[1]

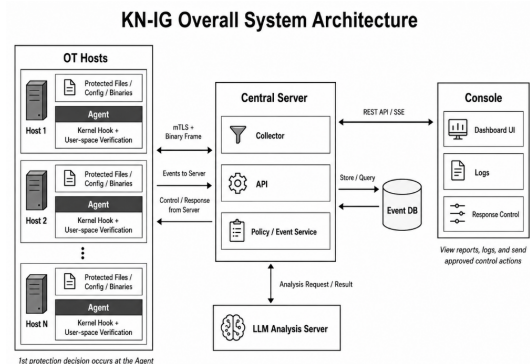
따라서 OT 환경의 요구사항에 맞는 체계적인 보안 정책과 가용성을 보장하는 차단 기능이 포함된 정책 기반 파일 무결성 보호 시스템이 필요하다.

II. KN-IG 설계 및 구조

본 연구의 무결성 보호 시스템 KN-IG의 보호 대상은 Linux 기반 OT 호스트에 존재하는 주요 설정 파일, 운영 폴더이다. 중요 파일의 비인가 변경은 적용 이전에 차단되어야 하며, 사후 대응을 위해 PID, UID, 경로, 시간 등의 행위 맥락이 보존되어야 한다. 또한 다양한 리눅스 커널 버전에서도 동일한 상위 정책 모델을 유지하고, 중앙 서버 장에 상황에서도 Agent가 독립적으로 1차 보호 판단을 수행할 수 있어야 한다.

그림 1. KN-IG 시스템 구조

Fig 1. Overall System Architecture of KN-IG



KN-IG의 전체 구조는 Agent, 중앙서버, 콘솔 기반 인터페이스, Database로 구성되며, 실제 보안 결정을 담당하는 중심은 Agent이다. Agent에서는 보

호 대상과 운용 모드를 로드한 뒤, 리눅스 커널 버전에 적합한 Loadable Kernel Module(이하 LKM) 혹은 eBPF 프로그램을 로드하여 파일 이벤트를 탐지하며 비인가 접근을 차단한다. 이후 탐지된 이벤트는 mTLS 암호화를 통해 중앙서버로 전송한다. 구조는 그림 1과 같다.

Agent는 이벤트 로그 생성 시 이벤트 발생 프로세스의 PID뿐만 아니라, 부모 프로세스 관계를 추적하여 PID chain을 함께 기록한다. 이를 통해 사후 조치를 위한 공격 행위 정보와 root cause를 확보할 수 있다.

Agent의 운용 모드는 lock과 maintenance로 구분된다. lock 모드는 정책 위반 시 즉시 차단을 수행하고 maintenance 모드는 동일 이벤트를 감시 대상으로 남기되 실제 변경은 허용한다. 이러한 이중 모드는 OT 환경에서 계획된 유지보수와 비인가 변경을 동일 규칙으로 처리할 수 없어 구분하였다.

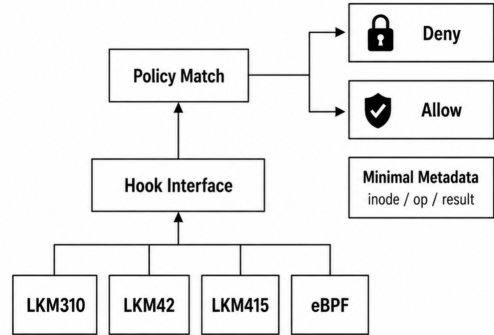
Agent는 운용 파일에 대한 Self-Protection을 수행하여 비인가 종료 및 설정 파일 변조 시도를 차단할 수 있다. 또한 kernel level hook을 통해 root 권한에서도 Agent 프로세스 종료를 막아, 중앙 서버에서 정책에 따른 절차를 통해 Agent 내부 프로세스 exit 명령을 수행하도록 하는 중앙 승인 종료 방식만을 허용하도록 한다.

정책 적용 대상 파일은 inode로 식별하도록 하였다. Agent 실행 시, 정책 적용 대상 파일을 유저랜드에서 순회하여 파일 경로와 inode, device Id를 추출하고 미리 저장하여 이후 파일 이벤트 발생 시 해당 파일에 대한 inode 비교만으로 정책 대상 파일 인지 즉시 판단할 수 있고, 파일 경로 추출 비용 또한 발생하지 않는다. 이는 커널 Hooking 시점부터 정책 기반 접근 제어를 완료하기까지의 지연 시간을 단축시키기 위한 설계이다.

커널 버전에 따라 사용 가능한 Hooking 인터페이스와 보안 메커니즘이 상이하므로, 본 연구에서는 모듈 호환성 확보를 위해 LKM310, LKM42, LKM415, eBPF LSM의 네 가지 구현을 구성하였다. 그림2는 서로 다른 커널 내부 인터페이스를 사용하지만, 동일한 inode 기반 정책 조회 및 차단 로직을 수행하도록 설계된 커널 구조를 나타낸다.

그림 2. 커널 동작 구조

Fig.2 kernel Operating Structure



구분	Kernel Version	차단 방식
LKM310	3.10~4.1	syscall hook
LKM42	4.2~4.14	LSM hook
LKM415	4.15~5.6	LSM hook
BPF LSM	5.8~	eBPF LSM hook

syscall 진입점을 sys_call_table을 통해 write, unlink, rename, truncate, open 계열 호출을 중재하고 정책 위반 행위가 수행되기 전 단계에서 이를 차단한다. inode 기반 LSM(Linux Security Module) hook은 inode 변조 차단을 통해 상위 보안 정책의 의미와 밀접하게 대응되어 시스템콜 인자 해석 시 발생하던 해석 차이의 격차를 최소화하였다 [4]. BPF LSM은 LSM Hooking 포인트에 eBPF 프로그램을 부착하여 MAC(Mandatory Access Control) 및 탐지 정책을 구현할 수 있는 인터페이스를 활용하여 Hooking Model 위에서 정책을 구동해 배포 용이성, 추적 가능성, 그리고 장기적 유지보수 이점을 확보하였다[5].

초기 Agent 등록 단계에서 제한된 물리 접근 또는 별도 절차를 통해 bootstrap 인증서를 배포할 수 있다. 이를 통해 초기 agent 등록과 인증서 발급을 수행하고, 이후 mTLS(mutual TLS) 기반 통신을 수행한다.

중앙서버는 이벤트를 수집하는 collector, API, 이벤트 저장 계층으로 구현되며, Agent와 collector 사이의 통신에는 텍스트 메시지 대신 파싱 비용을 줄이기 위해 고정 길이 헤더를 갖는 경량 이진 프레

임을 적용하였다. 또한 중앙 서버는 LLM Server에 데이터를 전송하여 분석 결과를 받고, 해당 결과에 따라 Agent에 자동 대응 명령을 보낸다.

콘솔 기반 인터페이스는 SSE와 REST API를 통해 LLM 보고서와 결과에 따른 대응과 차단 로그를 확인할 수 있는 대시보드 UI를 제공하며, Agent를 종료하는 기능을 제공한다.

III. 관련 연구와의 차이점

Wazuh와 같은 무결성 검사 시스템은 baseline 비교, 실시간 감시를 통해 파일 변경을 탐지하고 후속 대응을 수행한다[2],[3]. 그러나 이러한 방식은 변경 이후 탐지와 대응에 초점을 두어 연속성이 보장되어야 하는 OT 환경에는 적합하지 않다. 반면 KN-IG는 lock 모드에서는 write, delete, rename 계열 연산을 커널 혹은 레벨에서 차단하여 파일의 무결성을 보장한다.

후속 대응에서의 개선점은 LLM 서버를 통한 로그 분석과 사후 대응이다. LLM 서버에서 분석한 로그로 보고서를 작성하고 해당 보고서를 관리자가 검토하여 외부 침입인 경우를 판단하거나 LLM을 통해 자동 대응이 가능하다.

Linux-native integrity policy 계열인 IPE는 일반적인 무결성 정책에 초점을 둔다[6]. 이에 비해 KN-IG는 운영 파일 및 설정의 실시간 보호, 행위 맥락 수집, 데이터 수집을 하나의 Agent 구조로 통합한다.

IV. 검증

제안 시스템의 사전 차단 능력을 검증하기 위해 공격자는 임의의 시스템 콜 호출, 임시 파일 생성, 삭제, 프로세스 실행, rename/link 기반 우회 시도를 수행할 수 있다고 가정한 후 직접 write, truncate, chmod, unlink, rename base write bypass, hardlink, symlink 우회 등의 공격 시나리오를 구성하였다.

표 1은 구성한 7가지 시나리오에 대한 결과이다. lock 모드에서는 시스템 콜 단계에서 EPERM으로 차단되었으며, 보호 파일의 inode, 크기, 수정 시간이 유지 되었다. rename 기반 우회 시도는 새 파일

을 생성 뒤 이를 보호 파일 경로로 이동시켜 기존 파일을 덮어 쓰는 공격으로, 일반적인 경로에서 탐지가 어려운 시나리오이지만, path_rename hook에서 destination dentry를 검사하여 보호 대상 inode와 매칭되는 경우 작업을 차단하였다.

Link를 통한 write의 경우 hardlink와 symlink 생성 자체는 허용되었으나, 이를 통한 write 시도는 동일한 inode로 접근하기 때문에 file_open hook에서 정책이 매칭되어 차단된다. 이는 공격자가 접근 경로를 변경하더라도 inode 기반 정책을 우회하기 어렵다는 점을 보여준다. maintenance 모드에서 위험 시나리오의 변경 작업이 허용되었지만, 작업 종류에 대응하는 LSM hook 이름과 함께 탐지 로그가 기록되어 사후 분석을 위한 행위 추적이 가능함을 확인하였다.

표 1. 동작 검증 결과

Table 1. Operation verification results

위험 시나리오	Lock	LSM hook
직접 write	DENY	file_open
truncate	DENY	file_open
chmod	DENY	path_chmod
unlink	DENY	path_unlink
rename 우회	DENY	path_rename
hardlink write	write DENY	file_open
symlink write	write DENY	file_open

표 2. rename 기반 공격 시도 차단 로그

Table 2. denial log for rename-based bypass attempt

단계	내용	결과
1	보호 대상 파일의 초기 상태	inode=393274, BASELINE_TEXT
2	공격자가 보호 파일 복사	임시 파일 생성
3	공격자가 임시 파일 내용 변조	변조 완료
4	공격자가 변조된 임시파일을 보호 경로로 mv 수행	Operation not permitted
5	차단 후 보호 파일 상태	초기값에서 변화 없음
6	커널 감시 로그	path_rename hook : DENY

표 2는 rename 시나리오의 대표 실행 결과를 나타낸다. 공격자는 보호 파일을 임시 경로로 복사한

뒤 내용을 변조하고, mv 명령을 통해 보호 파일 경로로 다시 이동시켜 기존 파일을 덮어쓰려 하였지만, path_rename 혹에서 destination 경로의 inode가 보호 정책에 등록 되어 EPERM으로 거부하였다.

KN-IG의 Kernel 탑재로 인한 파일 I/O 오버헤드를 측정하기 위해 비활성/활성 상태를 비교하였다. open/close 마이크로벤치마크는 5회 반복하였고, fio 기반 4KB 시퀀셜 쓰기 부하는 30초 동안 3회 반복 수행하여 LKM은 단일 vCPU, 2GB RAM, eBPF는 4 vCPU, 3.8GB RAM Virtual Box에서 수행하였다.

표 3. 성능 측정 환경

Table 3. Performance Evaluation Environment

구분	OS	Kernel Version
LKM310	CentOS 7	Linux 3.10.0-1160.el...x86_64
LKM42	Ubuntu 16.04.7	Linux 4.4.0-186-generic
LKM415	Ubuntu 20.04.6	Linux 5.4.0-216-generic
eBPF LSM	Ubuntu 24.04.4	Linux 6.8.0-111-generic

표 4 KN-IG 비활성/활성 상태의 I/O 성능 비교

Table 4. I/O performance comparison with KN-IG disabled and enabled

모듈	항목	OFF	ON	변화량
lkm310	지연	1,912 ns	1,982 ns	+70 (+3.66%)
	처리량	4.37 MB/s	4.30 MB/s	-0.07 (-1.60%)
lkm42	지연	1,002 ns	999 ns	-3 (-0.32%)
	처리량	6.26 MB/s	6.17 MB/s	-0.09 (-1.45%)
lkm415	지연	1,348 ns	1,346 ns	-2 (-0.14 %)
	처리량	5.99 MB/s	5.98 MB/s	-0.01 (-0.12%)
eBPF	지연	1,906 ns	2,162 ns	+256 (+13.43 %)
	처리량	4.82 MB/s	4.57 MB/s	-0.25 (-5.19%)

표 5 KN-IG hooking 리소스 사용량

Table 5. Using Resource with KN-IG hooking

모듈	hooking	커널 메모리	RSS
lkm310	syscall hook	44 KB	19.9 MB
lkm42	LSM hook List	41 KB	11.7 MB
lkm415	LSM hook List	41 KB	17.0 MB
eBPF	LSM hook List	1,748 KB	49.8 MB

표 4는 각 모듈의 OFF/ON 상태에서 측정한 지연과 처리량의 비교 결과를 나타낸다. 지연은 open/close 1회 평균 소요 시간으로 측정하였고, 처리량은 fio 기반 순차 쓰기 부하에서의 초당 데이터 전송량으로 측정하였다. 측정 결과, eBPF는 다른 모듈에 비해 상대적으로 큰 지연 증가와 처리량 감소를 보였으며, 이는 eBPF LSM hook 실행과 정책 맵 조회 비용에 의한 오버헤드임을 확인할 수 있다. 특히 LKM42와 LKM415는 변화량이 매우 작아 측정 잡음 범위에 가까운 수준으로 나타났다.

표 5는 Agent의 Hooking 리소스 사용량을 나타낸 것으로 eBPF의 경우 LSM 모듈에 비해 커널 메모리 사용량이 약 42배 높고, RSS는 약 3배 더 높음을 알 수 있다. 이는 eBPF 자체 프로그램의 리소스 사용량이 크기 때문에 발생하는 차이이다.

V. 결론

본 논문은 Linux 기반 OT 환경을 위한 Agent 중심 정책 기반 파일 무결성 보호 시스템 KN-IG를 제안하였다. KN-IG는 inode 기반 정책 차단, kernel modules 기반 파일 접근 제어, mTLS 기반 데이터 수집을 하나의 보호 파이프라인으로 통합한다. 이 구조는 기존 범용 FIM의 post-change 탐지 중심 흐름을 OT 운영 연속성 요구에 맞추어 중요 파일 변경 차단, 행위 맥락 보존, 연속성 지향 설계로 확장한 것이다.

KN-IG는 향후 정량 성능평가와 대규모 OT 현장 검증이 더해질 때, 정책 기반 무결성 보호와 운영 연속성 보장을 함께 다루는 실용적 보안 체계로 발전할 수 있다.

참 고 문 헌

- [1] K. Stouffer et al., Guide to Operational Technology (OT) Security, NIST SP 800-82 Rev. 3, 2023;
M. Powell et al., Protecting Information and System Integrity in Industrial Control System Environments, NIST SP 1800-10, 2022.
- [2] Wazuh Documentation, “File Integrity Monitoring: Capabilities; How It Works,” accessed Apr. 15, 2026.
- [3] Wazuh Documentation, “Advanced Settings - File Integrity Monitoring: Active Response - Capabilities,” accessed Apr. 15, 2026.
- [4] Linux Kernel Documentation, “Linux Security Modules: General Security Hooks for Linux,” accessed Apr. 15, 2026.
- [5] Linux Kernel Documentation, “LSM BPF Programs,” accessed Apr. 15, 2026.
- [6] Linux Kernel Documentation, “Integrity Policy Enforcement (IPE),” accessed Apr. 15, 2026.
- [7] H. Lee et al., “KI-Mon: A Hardware-assisted Event-triggered Monitoring Platform for Mutable Kernel Object,” Proc. 22nd USENIX Security Symposium, 2013.
- [8] V. Gandhi et al., “Rethinking System Audit Architectures for High Event Coverage and Synchronous Log Availability,” Proc. 32nd USENIX Security Symposium, 2023.